

FORM VALIDATION

Step 1. Import React and useState

```
import React, { useState } from 'react'
```

Explanation

We import:

- React to create the component.
- useState to store data.

Think of useState as a box that stores values.

Example:

Name: Harinath
Email: abc@gmail.com
Phone: 9876543210

All these values need to be stored somewhere.

Step 2. Create the Component

```
const FormValidation = () => {
```

This is a functional component.

Everything related to this form is written inside this function.

Step 3. Create Form State

```
const [formData, setFormData] = useState({  
  name: "",  
  email: "",  
  phone: "",  
  password: "",  
  confirmPassword: ""  
})
```

What is State?

State stores data.

Initially,

formData

```
{  
  name:"",  
  email:"",  
  phone:"",  
  password:"",  
  confirmpassword:""  
}
```

Everything is empty because the user hasn't typed anything yet.

After typing

Suppose the user enters

Name : John
Email : john@gmail.com
Phone : 9876543210

Then

formData

```
{  
  name:"John",  
  email:"john@gmail.com",  
  phone:"9876543210",  
  password:"",  
  confirmpassword:""  
}
```

The state updates automatically.

Why do we use one object?

Instead of writing

```
const [name,setName]  
const [email,setEmail]  
const [phone,setPhone]
```

We keep everything inside one object.

This makes the code shorter and cleaner.

Step 4. Store Validation Errors

```
const [errors, setErrors] = useState({})
```

This object stores error messages.

Initially

```
errors = {}
```

No errors.

Suppose

Name is empty

Email is wrong

Then

errors

```
{  
  name:"Name is required",  
  email:"Invalid Email"  
}
```

Later we display these errors below the inputs.

Step 5. Regular Expressions

These are patterns used to check whether user input is correct.

Name Regex

```
const nameRegex = /^[A-Za-z]{3,30}$/
```

Meaning

^

Start

[A-Za-z]

Only letters

{3,30}

Minimum 3 letters

Maximum 30 letters

\$

End

Accepted

John

Ravi

Mahesh

Rejected

Jo

John123

Ravi@

Email Regex

```
const emailRegex =  
/^[^@\s]+@[^\s]+\.[^\s]+$/
```

Checks

abc@gmail.com

Not

abcgmail.com

abc@

@gmail.com

Phone Regex

```
const phoneRegex = /^[6-9]\d{9}$/
```

Meaning

Starts with

6

7

8

9

Then

9 more digits

Valid

9876543210
8123456789

Invalid

1234567890
5555555555
98765

Password Regex

```
const passwordRegex =  
/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/
```

This checks

✓ one lowercase

a-z

✓ one uppercase

A-Z

✓ one number

0-9

✓ one special character

@

\$
&

✓ minimum 8 characters

Example

Hello@123

Step 6. Handle Input Changes

```
const handleChange = (e) => {  
  
  setFormData({  
    ...formData,  
    [e.target.name]: e.target.value  
  })  
  
}
```

This is the most important function.

When the user types

John

React automatically sends an event.

e

Suppose

```
<input  
name="name"  
value={formData.name}  
onChange={HandleChange}  
>
```

Then

e.target.name

"name"

and

e.target.value

"John"

This line

[e.target.name]: e.target.value

becomes

name:"John"

Suppose

Current state

```
{  
  name:"",  
  email:"",  
  phone:""  
}
```

After typing

John

New state becomes

```
{  
  name:"John",  
  email:"",  
  phone:""  
}
```

What does

...formData

do?

It copies all existing values.

Without it

```
setFormData({  
  name:"John"  
})
```

State becomes

```
{  
  name:"John"  
}
```

Everything else disappears.

With

...formData

React keeps

Email
Phone
Password

and only updates

Name

Step 7. Validation Function

```
const validate = () => {
```

This function checks every input.

Create an empty object

```
let newErrors = {};
```

Initially

```
{}
```

Name Validation

```
if(!formData.name.trim()){
```

If user enters

```
"  "
```

trim() removes spaces.

```
"  "
```

↓

```
""
```

So this becomes empty.

Then

```
newErrors.name =  
"Name is required"
```

If user enters

John123

This condition

```
!nameRegex.test(formData.name)
```

returns

true

So

```
newErrors.name =  
"Only letters allowed"
```

Email Validation

Exactly same logic.

First

Empty?

Then

Regex?

Phone Validation

Same process.

Empty?

↓

Regex?

Password Validation

Same.

Empty?

↓

Regex?

Confirm Password

```
if(!formData.confirmpassword.trim())
```

Check empty.

Otherwise

```
formData.password !==  
formData.confirmpassword
```

Compare both passwords.

Example

Password

Hello@123

Confirm

Hello@123

Correct.

If

Hello@123

Hello123

Then

Passwords do not match

Step 8. Save Errors

```
setErrors(newErrors)
```

Suppose

Name missing

Email wrong

Then

```
errors
```

```
{  
  name:"Required",  
  email:"Invalid"  
}
```

React re-renders automatically.

Step 9. Return True or False

```
return Object.keys(newErrors).length === 0;
```

Suppose

```
{}
```

Number of keys

0

Returns

true

Validation passed.

Suppose

```
{  
  name:"Required",  
  email:"Wrong"  
}
```

Keys

2

Returns

false

Validation failed.

Step 10. Submit Function

```
const HandleSubmit = (e)=>{
```

Runs when the button is clicked.

Stop page refresh.

```
e.preventDefault();
```

Without this

The browser reloads the page.

All data disappears.

Call validation

```
if(validate())
```

If everything is correct

```
true
```

```
then
```

```
alert("Registration Successful")
```

After success

Reset state

```
setFormData({  
  name:"",  
  email:"",  
  phone:"",  
  password:"",  
  confirmpassword:""  
})
```

All fields become empty again.

Also remove errors.

```
setErrors({})
```

Step 11. JSX Form

```
<form onSubmit={HandleSubmit}>
```

When Submit button is clicked

↓

HandleSubmit() executes.

Name Input

```
<input  
  name="name"  
  value={formData.name}  
  onChange={HandleChange}  
>
```

Flow

User types



handleChange()



formData updates



Input displays new value

This is called a **Controlled Component** because React controls the input value through state.

Error Message

```
<p>{errors.name}</p>
```

If

errors.name="Name Required"

The page shows

Name Required

If no error exists

Nothing is displayed.

The same pattern is repeated for:

- Email
 - Phone
 - Password
 - Confirm Password
-

Complete Flow Diagram

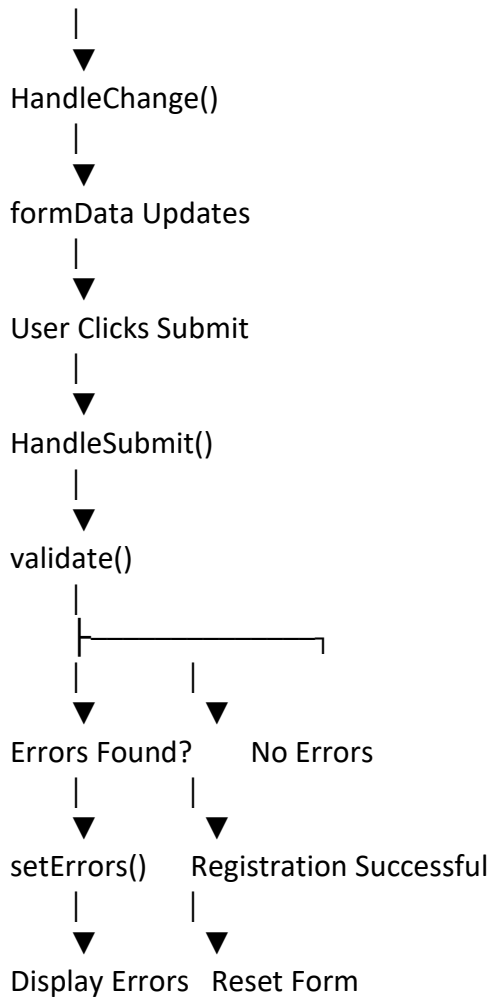
User Opens Form



All Inputs Empty



User Types



The 6 Step Formula for Form Validation

1. **Create form state** using `useState`.
2. **Create an errors state** to store validation messages.
3. **Write regular expressions** for fields like name, email, phone, and password.
4. **Handle input changes** with `onChange` and update the state.
5. **Validate the form** by checking empty fields, regex patterns, and matching passwords.
6. **Submit the form** only if validation succeeds, otherwise display the error messages below the respective fields.

This six step approach is reusable for almost every React form, such as registration forms, login forms, contact forms, profile forms, and checkout forms.

The remaining part is the **JSX (UI)**. This is where we connect the state, validation, and functions to the HTML form. Explain it to students from top to bottom.

Step 12. Return Statement

```
return (  
  <div>  
  
    <form onSubmit={HandleSubmit}>
```

What is return?

The return statement tells React what should be displayed on the screen.

Here React displays:

- Form
 - Input fields
 - Error messages
 - Submit button
-

Form

```
<form onSubmit={HandleSubmit}>
```

What does onSubmit do?

It listens for the Submit button click.

Flow:

User clicks Submit



HandleSubmit()



validate()



Success or Errors

Without this,

```
<form>
```

the form doesn't know what to do after clicking Submit.

Step 13. Name Input

```
<input  
  type="text"  
  name="name"  
  value={formData.name}  
  onChange={HandleChange}  
>
```

Let's understand every attribute.

type

`type="text"`

Creates a text box.

name

`name="name"`

This is very important.

This tells React

"This input belongs to the name property."

Remember `handleChange`?

`[e.target.name]: e.target.value`

Suppose

`name="name"`

Then

`e.target.name`

↓

`"name"`

So React updates

`formData.name`

If

`name="email"`

React updates

`formData.email`

This is why one `handleChange` function works for all inputs.

value

`value={formData.name}`

This connects the input box with state.

Initially

```
formData.name = ""
```

Textbox shows

Empty

After typing

John

State becomes

```
formData.name = "John"
```

Textbox also shows

John

React is controlling the textbox.

This is called a **Controlled Component**.

onChange

```
onChange={handleChange}
```

Whenever the user types

J

Jo

Joh

John

React calls

```
handleChange()
```

every single time.

Flow

User Types

↓

```
handleChange()
```

↓

setFormData()

↓

State Updates

↓

Textbox Updates

Step 14. Display Name Error

```
<p style={{ color: "red" }}>
  {errors.name}
</p>
```

Initially

```
errors = {}
```

Nothing appears.

Screen

Name Box

Suppose validation fails.

```
errors
```

```
{
  name:"Name is required"
}
```

React automatically displays

Name Box

Name is required

because

```
errors.name
```

contains a value.

If validation passes

errors.name

becomes

undefined

Nothing is displayed.

Step 15. Email Input

```
<input  
type="email"  
name="email"  
value={formData.email}  
onChange={HandleChange}  
>
```

Exactly same process.

Typing

abc@gmail.com

updates

formData.email

Display Error

```
<p>{errors.email}</p>
```

If invalid

Enter valid email

appears.

Step 16. Comment

```
{/* <h2>{formData.email}</h2> */}
```

This is just for testing.

If you remove the comment

```
<h2>{formData.email}</h2>
```

Whenever user types

abc@gmail.com

React displays

abc@gmail.com

below the textbox.

It is useful while learning state.

Step 17. Phone Input

```
<input  
  type="text"  
  name="phone"  
  value={formData.phone}  
  onChange={HandleChange}  
>
```

Same flow.

Typing

9876543210

updates

formData.phone

Error

```
<p>{errors.phone}</p>
```

Step 18. Password Input

```
<input  
  type="password"  
  name="password"  
  value={formData.password}  
  onChange={HandleChange}  
>
```

Difference

password

shows

instead of

Hello@123

for security.

Display Error

<p>{errors.password}</p>

Step 19. Confirm Password

```
<input
  type="password"
  name="confirmpassword"
  value={formData.confirmpassword}
  onChange={HandleChange}
/>
```

Typing

Hello@123

updates

formData.confirmpassword

Error

<p>{errors.confirmpassword}</p>

Step 20. Submit Button

```
<button type="submit">
  Submit
</button>
```

When clicked

Button

↓

HandleSubmit()

↓

preventDefault()

↓

validate()

↓

Success / Errors

Step 21. Closing Tags

```
</form>
```

```
</div>
```

```
)
```

Nothing special.

Just closing JSX elements.

Step 22. Export Component

export default FormValidation

Why?

Without exporting

FormValidation

cannot be used inside

App.jsx

With export

```
import FormValidation from "../FormValidation";
```

React can display the component.

Complete Flow of the Program

Application Starts



States Created



Form Appears



User Types



HandleChange()



formData Updates



User Clicks Submit



HandleSubmit()



preventDefault()



validate()



Validation Failed



setErrors()

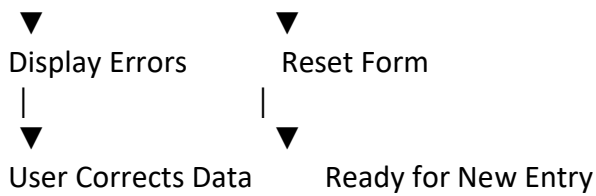


Validation Passed



Alert Success





The Most Important Concepts Should Remember

Concept	Purpose
useState	Stores form data and errors
formData	Holds all input values
errors	Holds validation messages
handleChange()	Updates state whenever the user types
Regex	Checks whether input follows the required pattern
validate()	Validates every field and fills the errors object
handleSubmit()	Prevents page refresh, runs validation, and submits if valid
value={formData...}	Makes the input a controlled component
onChange={handleChange}	Calls the update function on every keystroke
onSubmit={handleSubmit}	Runs when the form is submitted
errors.fieldName	Displays the corresponding validation message
export default	Makes the component available to use in other files